

Python Automata Library (PAL) Cheat Sheet

```
import PythonAutomataLibrary as pal · Safe
mode is default. Call pal.FastMode() before
constructing any PAL object. Put substantive model
kernels under @pal.njit(cache=True).
```

Create state

`pal.NewGrid(dims, dtype)` — typed lattice
`pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False)` — individual agents;
`dims=()` is nonspatial
`pal.NewPopGrid(dims, capacity=None)` — integer population counts
`pal.NewPDEgrid(dims)` — continuous field
`pal.NewIList()` · `pal.NewMultinomial()` — integer query list · random-count sampler

Dimensions — 1–3 axes; a negative dimension wraps that axis, e.g. `(-nx, ny)`.

Shared lattice geometry

Properties — `xDim/yDim/zDim, nDims, wrapX/Y/Z`

Index conversion — `ToI(x[,y,z])`; `ItoX/Y/Z(i)`

Regions — `Box(lo..., hi...)` uses half-open bounds; `Hood(hood, x[,y,z])` maps relative offsets to coordinates.

Neighborhoods — `pal.MooreHood(dim, includeOrigin)` · `pal.VonNeumannHood(dim)` · `pal.CircleHood(dim, rad)`

Grid / common indexing

Read/write — `g[x,y]`, `g[x,y]=v`; slices return detached NumPy copies.

Pattern — `g = pal.NewGrid((40,40), float)` · `g[10,12] = 1.0 · v = g[10,12]`

AgentGrid

Create / move — `NewAgentSQ(x,y)` / `MoveSQ(a,x,y)` for lattice positions; `NewAgent(x,y)` / `Move(a,x,y)` for continuous positions.

Agent state — `grid[a,p]` property; `I(a)`, `XSQ/YSQ/ ZSQ(a)` lattice; `X/Y/Z(a)` continuous; `Alive(a)`, `Dispose(a)`.

Queries — `GetPop()`, `AgentsAt(...)`, `LastAgent(...)`, `AgentsInRadius(...)`, `counts[...]`.

Iteration — for `a` in `agents.All()`: ... is a snapshot and is safe for structural mutation such as `Dispose`.

Wrapping — `DispWrapX/Y/Z(p1,p2)` gives wrapped displacement.

PopGrid + PDEgrid

Transactional update — `Add(v, ...)` changes pending state; `Update()` applies it simultaneously; `Reset()` clears current + pending. `Direct grid[...] = v` changes current state immediately.

Population — `pop.GetPop()` total; `pop.All()` nonzero site indices. capacity limits total population.

PDE setup — `field.SetTimeSpaceStep(dt, dx[,dy,dz])`

Diffusion — `Diffusion`, `DiffusionMask`, `DiffusionField`, `DiffusionInterfaces`, `DiffusionADI`; radial 1D: `DiffusionRadialCircle/Sphere`.

Advection — `Advection`, `AdvectionField`, `AdvectionInterfaces`.

Lists + randomness

IList — `Append(i)`, `Clear()`, `Random()`, `Shuffle()`, `indexing/len`; `All()` detached copy; `Iter()` no-copy iteration.

RNG — `pal.Seed(seed)` · `pal.Random()` · `pal.RandInt(n) → 0..n-1`; use PAL RNG for shared-stream reproducibility.

Multinomial — `m.Binomial(n,p)`; `Setup(...)` then `Sample(...)` for repeated multinomial draws.

Draw / output

Pixels — `pix, win = pal.StartPixWindow(xDim,yDim,scale=1,title='PAL',headless=False)`; `set pix[x,y]=RGB`; `win.Update()`; `win.Save(path, block=True)`; `win.Close()`.

GIF — `StartGif(path,delay=100)` · `AddGifFrame(block=False)` · `StopGif()`; call `Update()` before capture.

OpenGL — `pal.StartOpenGLWindow(...)`; draw with `Circle`, `Box`, `BoxSQ`, `Line`, `Borders`; scene controls include `Camera`, `Background`, `Clear`.

More detail: [Manual](#) · [API Guide](#) · [API Reference \(Documentation/\)](#).